

Database Documentation — EbookStore

Teknologi

- **Database Engine:** SQLite (via `sqflite` package)
- **Database Name:** `ebookstore.db`
- **Current Schema Version:** 5

Daftar Tabel

Tabel	Deskripsi
<code>categories</code>	Kategori buku (dikelola Admin)
<code>users</code>	Pengguna (buyer, seller, admin)
<code>books</code>	Buku yang dijual oleh seller
<code>wishlist</code>	Buku favorit yang disimpan buyer
<code>cart</code>	Keranjang belanja buyer
<code>orders</code>	Pesanan yang dibuat buyer

Relasi Foreign Key

1. `books.kategori` → `categories.nama`

```
FOREIGN KEY (kategori) REFERENCES categories(nama)
ON DELETE RESTRICT ON UPDATE CASCADE
```

Tipe Relasi	Many-to-One (Many-to-Many jika dibalik)
-------------	---

- Satu **kategori** dapat dimiliki oleh **banyak buku**.
- Setiap **buku** hanya memiliki **satu kategori**.
- `ON DELETE RESTRICT` → kategori tidak bisa dihapus jika masih ada buku yang menggunakannya.

- `ON UPDATE CASCADE` → jika nama kategori diubah, semua buku dengan kategori tersebut otomatis mengikuti.
-

2. `books.seller_id` → `users.id`

```
FOREIGN KEY (seller_id) REFERENCES users(id)
ON DELETE SET NULL ON UPDATE CASCADE
```

Tipe Relasi	Many-to-One
-------------	-------------

- Satu seller dapat memiliki **banyak buku**.
 - Setiap buku hanya memiliki **satu seller** (atau `NULL` jika seller dihapus).
 - `ON DELETE SET NULL` → jika seller dihapus, `seller_id` pada buku menjadi `NULL` (buku tidak ikut terhapus).
 - `ON UPDATE CASCADE` → jika `id` seller berubah, `seller_id` di buku otomatis mengikuti.
-

3. `wishlist.user_id` → `users.id`

```
FOREIGN KEY (user_id) REFERENCES users(id)
ON DELETE CASCADE ON UPDATE CASCADE
```

Tipe Relasi	Many-to-One
-------------	-------------

- Satu user dapat memiliki **banyak wishlist item**.
- Setiap **wishlist item** hanya dimiliki oleh **satu user**.
- `ON DELETE CASCADE` → jika user dihapus, semua wishlist item-nya ikut terhapus.

4. `wishlist.book_id` → `books.id`

```
FOREIGN KEY (book_id) REFERENCES books(id)
ON DELETE CASCADE ON UPDATE CASCADE
```

Tipe Relasi	Many-to-One
-------------	-------------

- Satu buku dapat ada di **banyak wishlist** (dari user berbeda).
- Setiap **wishlist item** hanya merujuk ke **satu buku**.
- `ON DELETE CASCADE` → jika buku dihapus, semua wishlist yang merujuk buku tersebut ikut terhapus.

5. `cart.user_id` → `users.id`

```
FOREIGN KEY (user_id) REFERENCES users(id)
ON DELETE CASCADE ON UPDATE CASCADE
```

Tipe Relasi	Many-to-One
-------------	-------------

- Satu user dapat memiliki **banyak cart item**.
- Setiap **cart item** hanya dimiliki oleh **satu user**.
- `ON DELETE CASCADE` → jika user dihapus, semua cart item-nya ikut terhapus.

6. `cart.book_id` → `books.id`

```
FOREIGN KEY (book_id) REFERENCES books(id)
```

```
ON DELETE CASCADE ON UPDATE CASCADE
```

Tipe Relasi	Many-to-One
-------------	-------------

- **Satu buku** dapat ada di **banyak cart** (dari user berbeda).
- **Setiap cart item** hanya merujuk ke **satu buku**.
- `ON DELETE CASCADE` → jika buku dihapus, semua cart yang merujuk buku tersebut ikut terhapus.

7. `orders.buyer_id` → `users.id`

```
FOREIGN KEY (buyer_id) REFERENCES users(id)  
ON DELETE RESTRICT ON UPDATE CASCADE
```

Tipe Relasi	Many-to-One
-------------	-------------

- **Satu buyer** dapat memiliki **banyak order**.
- **Setiap order** hanya dimiliki oleh **satu buyer**.
- `ON DELETE RESTRICT` → user tidak bisa dihapus jika masih memiliki order (menjaga riwayat transaksi).

8. `orders.book_id` → `books.id`

```
FOREIGN KEY (book_id) REFERENCES books(id)  
ON DELETE RESTRICT ON UPDATE CASCADE
```

Tipe Relasi	Many-to-One
-------------	-------------

- **Satu buku** dapat muncul di **banyak order**.
- **Setiap order** hanya merujuk ke **satu buku**.
- `ON DELETE RESTRICT` → buku tidak bisa dihapus jika sudah ada order (menjaga integritas riwayat transaksi).

Ringkasan Relasi

One-to-Many (1 : N)

Parent (1)	Child (N)	Melalui Field
categories	books	books.kategori
users (seller)	books	books.seller_id
users (buyer)	wishlist	wishlist.user_id
books	wishlist	wishlist.book_id
users (buyer)	cart	cart.user_id
books	cart	cart.book_id
users (buyer)	orders	orders.buyer_id
books	orders	orders.book_id

Many-to-Many (N : N)

Tabel Pertama	Tabel Kedua	Tabel Penghubung	Foreign Keys
users	books	wishlist	wishlist.user_id → users.id, wishlist.book_id → books.id
users	books	cart	cart.user_id → users.id, cart.book_id → books.id

Penjelasan: Relasi Many-to-Many antara `users` dan `books` terjadi secara tidak langsung melalui tabel perantara (`wishlist` dan `cart`). Satu user dapat memiliki banyak buku di wishlist/cart-nya, dan satu buku dapat dimiliki oleh banyak user di wishlist/cart mereka. Karena SQLite relasional tidak mendukung M:N secara langsung, kita gunakan tabel penghubung dengan dua Foreign Key.

Penjelasan Detail Relasi

One-to-Many (Satu ke Banyak)

Definisi: Satu record di tabel A berhubungan dengan banyak record di tabel B, tetapi satu record di tabel B hanya berhubungan dengan satu record di tabel A.

Contoh pada aplikasi:

- **Kategori → Buku:** Kategori "Fiksi" bisa memiliki puluhan buku. Namun setiap buku hanya bisa memiliki satu kategori (misal: "Fiksi").
- **Seller → Buku:** Seorang seller bisa menjual banyak buku. Namun setiap buku hanya dimiliki oleh satu seller.
- **Buyer → Order:** Seorang buyer bisa melakukan banyak pesanan. Namun setiap pesanan hanya milik satu buyer.

Ciri khas dalam skema:

- Foreign Key ditempatkan di **tabel Child (N)** yang merujuk ke Primary Key di **tabel Parent (1)**.
- Contoh: `books.seller_id` (child) → `users.id` (parent).

Many-to-Many (Banyak ke Banyak) via Tabel Penghubung

Definisi: Satu record di tabel A berhubungan dengan banyak record di tabel B, dan sebaliknya satu record di tabel B juga berhubungan dengan banyak record di tabel A.

Contoh pada aplikasi:

- **User ↔ Buku (via Wishlist):** Seorang user bisa menyimpan banyak buku favorit. Sebuah buku juga bisa difavoritkan oleh banyak user.
- **User ↔ Buku (via Cart):** Seorang user bisa memasukkan banyak buku ke keranjang. Sebuah buku juga bisa dimasukkan ke keranjang oleh banyak user.

Ciri khas dalam skema:

- Dibutuhkan **tabel perantara (junction table)** yang berisi dua Foreign Key.
- Tabel perantara (`wishlist`, `cart`) tidak memiliki makna bisnis sendiri; fungsinya hanya menghubungkan.
- Kombinasi kedua FK biasanya dibuat `UNIQUE` untuk mencegah duplikasi.

Perbandingan

Aspek	One-to-Many	Many-to-Many
Jumlah tabel	2 tabel	3 tabel (dengan junction table)
Letak FK	Di tabel Child (sisi N)	Di junction table (dua FK)
Contoh real	Satu kategori → banyak buku	Banyak user ↔ banyak buku (via wishlist)
Batasan kardinalitas	Setiap child hanya terhubung ke satu parent	Tidak ada batasan, semua arah bebas N:N

Catatan Penting

1. ``PRAGMA foreign_keys = ON`` diaktifkan setiap kali database dibuka (`onOpen`) agar SQLite benar-benar menegakkan constraint FK.
2. **Urutan pembuatan tabel** di `_createAllTables()` mengikuti dependency: parent table dibuat terlebih dahulu sebelum child table.
3. **Urutan penghapusan tabel** di `_onUpgrade()` adalah kebalikannya: child table dihapus terlebih dahulu.
4. **Seed data** di-insert ulang setiap kali versi database berubah (destructive upgrade).